[F26] Introduction to Computer Vision

Food Image Classification

Project Plan

The project studies food image classification on a selected subset of Food-101. The main experiments evaluate the effect of ImageNet pretraining and data augmentation on classification quality and on errors between visually similar food classes.

Dataset
Food-101

Main model
ResNet-18

Main experimental factors
ImageNet pretraining and data augmentation

October 2026

Project Scope

Objective

The goal of the project is to build and evaluate an image classifier for food recognition. The project will use a subset of 12 classes from the Food-101 dataset:

- carrot cake
- cheesecake
- chocolate cake
- red velvet cake
- ramen
- pho
- spaghetti bolognese
- spaghetti carbonara
- fried rice
- risotto
- hamburger
- pizza

Class Selection

Several visually similar classes are included on purpose. The selected subset contains multiple cakes, noodle dishes, pasta dishes, and rice dishes.

This makes it possible to measure not only overall classification quality, but also whether the model systematically confuses classes with similar appearance, ingredients, texture, or presentation.

Pizza and hamburger are included as more visually distinctive classes. They provide a useful comparison with the more difficult groups and help show whether the errors are concentrated mainly among similar classes.

Model and Experimental Factors

The main model will be **ResNet-18**.

The project will study two factors:

- ImageNet pretraining
- data augmentation

The same model architecture will be used in all main experiments. This keeps the architecture constant and makes it possible to measure the effect of pretraining and augmentation directly.

Research Question

How do ImageNet pretraining and data augmentation affect classification performance on a subset of Food-101 that contains visually similar food classes?

Dataset Preparation

Dataset

The project will use the public Food-101 dataset. Only the 12 selected classes will be included in the experiments.

Food-101 provides 750 training images and 250 test images for each class.

The original test set will remain unchanged. The original training set will be divided into 600 training images and 150 validation images per class.

The final dataset will contain:

- **7,200** training images
- **1,800** validation images
- **3,000** test images
- **12,000** images in total

The same split will be used in every experiment. This keeps the evaluated data constant, so differences between experimental conditions are not caused by different samples.

Image Preprocessing

All images will be prepared for the 224 by 224 input size used by ResNet-18.

For experiments without augmentation, preprocessing will include:

- resizing
- center cropping to 224 by 224 pixels
- conversion to tensors
- normalization

Data Augmentation

For experiments with augmentation, training images will use:

- random resized cropping to 224 by 224 pixels
- random horizontal flipping
- small random color changes
- conversion to tensors
- normalization

These transformations create additional variation in viewpoint, framing, and color while preserving the food class. This allows the model to see more varied versions of the available training images.

Validation and test images will use fixed preprocessing so that evaluation does not depend on random image transformations.

Experimental Plan

Experimental Conditions

Four conditions will be compared:

- ResNet-18 with random initialization and no augmentation
- ResNet-18 with ImageNet pretraining and no augmentation
- ResNet-18 with random initialization and augmentation
- ResNet-18 with ImageNet pretraining and augmentation

The first condition will serve as the baseline.

Comparing the first and second conditions will measure the effect of ImageNet pretraining.

Comparing the first and third conditions will measure the effect of data augmentation.

The fourth condition will show how pretraining and augmentation perform when used together.

Training Strategy

For pretrained models, the complete ResNet-18 network will be fine tuned.

This allows both the final classifier and the feature extraction layers to adapt from general ImageNet features to features that are useful for food recognition.

The planned training settings are:

- AdamW optimizer
- initial learning rate of 3×10^{-4}
- batch size of 32
- maximum of 30 epochs
- learning rate scheduling during training
- model selection based on validation accuracy

AdamW provides weight decay during optimization and can be used consistently for both randomly initialized and pretrained models.

A batch size of 32 provides a practical balance between memory use and the number of samples used for each gradient update.

The same training settings will be used across the four conditions so that the main difference between experiments remains the use of pretraining and augmentation.

Repeated Experiments

Each condition will be trained three times with different random seeds.

Repeated runs reduce the influence of random initialization, training sample order, and random augmentation on the final comparison.

The final results will therefore be reported using the **mean** and **standard deviation** across the three runs.

Evaluation Metrics

The main evaluation metrics will be:

- **Accuracy**. The selected classes contain the same number of images, so accuracy provides a direct measure of overall classification performance.
- **Macro F1 score**. The F1 score will be calculated separately for every class and then averaged. This helps identify whether strong overall performance hides weaker results for individual classes.

Additional analysis will include:

- precision for each class
- recall for each class
- confusion matrix
- analysis of the most common confusion pairs
- manual inspection of representative classification errors

Confusion and Failure Analysis

The confusion matrix will be used to identify which classes are most frequently confused.

This analysis is especially important for the selected groups of visually similar foods, such as cakes, noodle dishes, pasta dishes, and rice dishes.

Around six representative errors will be inspected manually.

Possible causes will include:

- similar ingredients
- similar shape or texture
- similar presentation
- unusual viewing angles
- background objects
- partial visibility of the food
- images that are visually ambiguous even for a human observer

The test set will be used only after training and model selection are complete. Validation data will be used for checkpoint selection and training decisions. The test set will provide the final evaluation.

Implementation Plan

Machine Learning Stack

The main machine learning stack will consist of:

- Python
- PyTorch

- torchvision
- scikit-learn
- pandas
- Matplotlib

PyTorch will be used for model training and inference.

Torchvision provides the Food-101 dataset interface, the ResNet-18 implementation, ImageNet pretrained weights, and the image transformations required for preprocessing and augmentation.

Scikit-learn will be used for classification metrics and confusion matrix calculation.

Pandas will be used to organize and compare experimental results.

Matplotlib will be used to create result visualizations.

Model Development

The implementation will include the following steps:

- prepare the selected Food-101 classes
- create the fixed training and validation split
- implement preprocessing and augmentation
- configure ResNet-18 for 12 output classes
- train all four experimental conditions with three random seeds
- save model checkpoints and training results
- evaluate the trained models on the test set
- calculate the main metrics and metrics for each class
- generate the confusion matrix
- collect misclassified samples
- aggregate the results from repeated runs
- select the final model based on the experimental results

Training and evaluation will be implemented as Python programs. This format is suitable for repeated machine learning experiments because the same training and evaluation procedure can be executed with fixed configurations and random seeds.

Prediction Backend

The prediction backend will use **FastAPI**.

FastAPI allows the trained PyTorch model and the image preprocessing pipeline to be used directly inside the Python backend while providing an HTTP interface for the frontend.

The backend will:

- load the final trained model when the application starts
- receive an image
- validate the input image
- apply the same preprocessing used during evaluation
- run model inference

- return the predicted class and confidence score

Frontend

The frontend will be implemented with **React**.

It will provide:

- image upload
- image preview
- communication with the backend
- predicted food class
- prediction confidence
- loading states
- error handling

Infrastructure

The project infrastructure will use:

- GitHub
- Docker
- GitHub Actions

Docker will provide a consistent environment for the application and its dependencies.

GitHub Actions will automatically check that the project can be installed, tested, and built correctly after changes to the repository.

The experiment setup will also preserve the configurations and random seeds required to reproduce the reported training and evaluation results.

Team Responsibilities

Denis Geraskin

Email: d.geraskin@innopolis.university

Area: AI Engineering and Backend

Responsibilities:

- implementation and configuration of ResNet-18
- implementation of random and ImageNet pretrained initialization
- training and fine tuning of the model
- execution of the main experimental conditions
- model checkpoint handling
- implementation of model inference
- model related backend logic

Denis Beliaev

Email: d.beliaev@innopolis.university

Area: Data Science, Data Preparation and Backend

Responsibilities:

- preparation of the Food-101 subset
- creation of the training, validation, and test split
- implementation of preprocessing and augmentation
- implementation of quantitative evaluation
- calculation of Accuracy, Macro F1, precision, and recall
- generation and analysis of the confusion matrix
- aggregation of results from repeated runs
- identification of important confusion patterns
- backend image preprocessing and validation

Aiaz Nadirov

Email: a.nadirov@innopolis.university

Area: Infrastructure and Deployment

Responsibilities:

- GitHub repository organization
- dependency and environment management
- Docker configuration
- GitHub Actions configuration
- automated testing
- deployment of the final application
- reproducible execution of experiments
- preparation of setup, training, evaluation, and launch instructions

Marat Yunusov

Email: mar.yunusov@innopolis.university

Area: Frontend and Qualitative Analysis

Responsibilities:

- implementation of the React frontend
- communication between the frontend and backend
- image upload and prediction display
- interface states and error handling
- integration testing of the complete application
- preparation of result visualizations
- manual inspection of classification errors

- grouping representative failures by their likely visual cause
- preparation of representative prediction examples for the final demonstration